Guidelines for Linux Installation

Bastien Chassagnol, Benjamin Loire

Table of contents

1 Linux general guidelines **2**
1.1 General Folder layout . . . 2
1.1.1 System-wide location (for core programs) . . . 2
1.1.2 User-specific locations . . . 2
1.1.3 General remarks . . . 3
1.2 Useful Ubuntu commands . . . 3
1.3 General update of Ubuntu . . . 3
1.4 Network issues . . . 4
1.5 Eduroam connexion . . . 5

2 Git and GitHub **5**
2.1 Github configuration . . . 5

3 Synchronise files across devices **6**
3.1 Serverless . . . 6
3.2 OneDrive and Nextcloud . . . 6
3.2.1 Troubleshoots for OneDrive . . . 9
3.3 Monitor processes in the long-run . . . 11

4 Software to install **11**
4.1 Shell-customisation (including `$PATH`) . . . 13
4.1.1 Zsh (advanced bash configuration) . . . 13
4.1.2 Add executables . . . 13
4.2 AppImageLauncher . . . 14
4.3 PDF management . . . 14
4.3.1 Sioyek as PDF Viewer . . . 14
4.3.2 Sejda for editing PDFs in an interactive way . . . 14
4.3.3 PDFtk . . . 14
4.4 Image Editors (to replace Paint on Windows) . . . 14
4.5 LibreOffice . . . 15
4.6 Docker . . . 15
4.7 Zoom . . . 16
4.8 Teams . . . 16
4.9 Firefox . . . 16

4.10 Slack 17
4.11 Zulip 17
4.12 Cytoscape 17
4.13 Zotero 18
4.14 Quarto, R and RStudio 19
4.15 Latex 19
4.16 WhatsApp 20
4.17 Cursor 20

1 Linux general guidelines

1.1 General Folder layout

1.1.1 System-wide location (for core programs)

System-wide affects all users, are installed using `sudo`, on the root of the system.

Location	Purpose
`/usr/bin`	System executables (APT)
`/etc`	System-wide configuration (example: define repositories for `apt`)
`/opt`	Vendor (external) apps
`/usr/local/bin`	Admin-installed software
`/var`	Logs, caches, databases

Table 1: Main system-wide software locations

- `/opt` for third-party application bundles, **self-contained** while `/usr/local` is for third-party **binaries**. Besides, `/opt` is not in the path by default.

1.1.2 User-specific locations

User does not require root

Location	Purpose
`~/Applications`	AppImages
`~/.local/bin`	User executables
`~/scripts/tools`	Custom bash executables
`~/.local/share`	User app data, notably `~/.local/share/applications` storing desktop icons
`~/.config`	User configuration

Location	Purpose

Table 2: Main user-based software locations, to be compared with Table 1

1.1.3 General remarks

- `usr` folder stores *read-only* executables, while `/var` is regularly written upon changes:
  - `/var/lib`: apt and dpkg databases, for instance, respectiely stored under `dpkg`, and `/apt`
  - `/var/cache`: metadata caches for faster execution
- `/usr/local/`, or `$HOME/.local` stores executables that require custom compilation with `make`, or `qmake`, rendered from manual archives:
  - `bin`, `sbin` for executables
  - `share` for icons (under subfolder `icons`) and desktop configuration, documention under `/doc`
  - `lib` for extensions of a given executable (such as packages in R, or modules in Python)
  - This folder hierachy follows `XGD`Base Directory Npecification
- `/tmp` for temporary folders, cleaned after `reboot`
- `/run` for sockets

1.2 Useful Ubuntu commands

- `dpkg --print-architecture`: retrieve architecture, for package installation[1].
- `Ctrl-H` in **Gnome**, aka File Management, to list hidden files

1.3 General update of Ubuntu

1. Create file `update_os.sh` with:

```
#!/bin/bash
rm -rf /var/lib/dpkg/lock-frontend
rm -rf /var/lib/dpkg/lock
apt-get update
apt-get upgrade -y
apt-get dist-upgrade -y
apt-get autoremove -y
apt-get autoclean -y
```

2. Run having `sudo`rights

[1] Returns `amd64` in my case

i. `chmod +x update_os.sh` to provide executive rights
ii. `sudo ./update_os.sh` to run the update file with **admin** rights

3. Reboot the system with `reboot`

1.4 Network issues

- Two IP protocols for Wi-FI: `IPv4` versus `IPv6`.

> The two Internet protocols, namely IPv4 vs IPv6
>
> IPv4 is older, but still prevailing in many websites, such as Github, or OneDrive:
> - Uses 32-bit addresses (e.g. `192.168.1.10`)
> - Used by **most websites**, CDNs, VPNs, corporate services
> - But the 32-limitation hinders the number of avalaible IP addresses.
>
> IPv6 is newer, but not universally supported:
> - Uses 128-bit addresses (e.g. `2a01:cb14:86a7::1`)
> - Supported by modern ISPs and operating systems
> - Some networks provide **IPv6-only**, with no IPv4 fallback. Espaically the case with Microsoft services, governmental or academic websites.

- You can test if this is the issue to connect to some websites with `curl <webdomain>`, such as `curl https://onedrive.live.com`, returning *Cannot connect*

- Recent modern routers return only IPv6 -> while Windows is able to able to auto-fall back to IPv4, this is not the case of IPv6.

  - *Temporary fix*: by running the following instructions, you solve this issue by automatically configuring a IPv4 upon failure. *Recommended only for short-term debugging.*

```
install dhclient commands
sudo dhclient -4 wlp2s0f0 <Wifi-name> # in my case, wlp2s0f0
```

  - *Permanent fix*[2]:

```
nmcli connection modify "Routeur/Livebox-FC77" ipv4.method auto
nmcli connection modify "Routeur/Livebox-FC77" ipv6.method auto # connect automatic
# re-start the connection, applying the new configuration parameters
nmcli connection down "Routeur/Livebox-FC77"
nmcli connection up "Routeur/Livebox-FC77"
```

[2] Alternatively, you can rely on the GNOME Interface, `Wifi` section

1.5 Eduroam connexion

0. **Ensure to delete beforehand any pre-existing installation: `rm -rf $HOME/.config/cat_installer`.**
1. Go to https://cat.eduroam.org/, the website usually provides with the version fitting your OS distribution.
2. On Linux, download the Python script, choosing AMU university.
3. Run Python script with `python 3 eduroam-linux-AUA.py`
4. As `userid`, provide your academic mail address, and as password, your `ENT password`.

2 Git and GitHub

2.1 Github configuration

1. Github often requires double authentication, that you can set on https://github.com/settings/security. Recommended to use *passkey* if supported by your device (unfortunately, Linux OS usually do not support it), or use authenticator app, such as Microsoft Authentificator. **Deterred against the use of SMS + mobile phones**.

2. Configure your access to Github:

i. Choose either SSH, or HTTPs.
ii. For HTTPS, configure one token per device, under https://github.com/settings/tokens. Give the token the name of your laptop, its configuration and originals owner. Assign it at least the repo and read scopes. Copy it once -> **it won't be available afterwards!!**.

3. General options to configure:

i. Use either the command line (create also automatically the file `~/.gitconfig`):

```
git config --global user.name "bastienchassagnol"
git config --global user.email "bastien.chassagnol@univ-amu.fr"
```

ii. Or directly, using text editor, file `~/.gitconfig` (more error-prone to indentation errors). Find my default git config file here (for comprehensive description of options, report to Formation Git 2024, a tutorial by Benjamin):

```
[user]
  name = bastienchassagnol
  email = bastien.chassagnol@univ-amu.fr
# modern default git options
[init]
  defaultBranch = main
```

```
# simplifying solving git conflicts + troubleshoot solving
[merge]
  conflictstyle = zdiff3
[rerere]
  enabled = true

# tolerate distinct end of lines
[core]
  autocrlf = input

# visualisation
[tag]
  sort = version:refname
[branch]
  sort = -committerdate
[credential]
  helper = store
```

4. Avoid typing your password for every login:

   i. For HTTPS + PAT token protocol, use `git config --global credential.helper store` for **permanent** registration of the password (should we not encrypt the file??), or `git config --global credential.helper 'cache --timeout=3600'` for temporary access[3].
   ii. Define SSH key.

3 Synchronise files across devices

3.1 Serverless

- Go for `Syncthing`.

3.2 OneDrive and Nextcloud

> If you already have a monthly pass, and are happy with onlien edition of the Office toolkit, go for it. Alternatively, `Nextcloud` also allows for interactive use of the Office toolkit, or `Rclone` for syncing files across computers.

1. Install dependencies:

```
sudo apt install build-essential libcurl4-openssl-dev pkg-config git -y
```

2. Add the `OpenSuSE` Build Service repository release key

[3] Time-out units are reported in seconds, accordingly, this amounts to one hour

```
wget -qO - https://download.opensuse.org/repositories/home:/npreining:/debian-ubuntu-onedriv

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/obs-onedrive.gpg]

# update cache
sudo apt-get update
```

3. Install `OneDrive`:

```
sudo apt install --no-install-recommends --no-install-suggests onedrive
```

4. Sync OneDrive:

i. Exclude large folders, and directories that are complex and are not recommended for syncing, `nano ~/.config/onedrive/config`

My config file contains:

```
# Exclude some configuration folders
## Python virtual environments
skip_dir = "venv|\.venv|__pycache__"
## Node.js / JavaScript build directories
skip_dir = "node_modules|\.next"
## R virtual environments
skip_dir = "renv|.cache|.Rproj.user"
skip_file = "\.Rhistory$|\.RData$"
## General files not to sync
skip_file = "~*|.~*|*.tmp|*.swp|*.partial"
# No sync the folder, if .nosync file is present
check_nosync = "true"
# local folders take precedence over remote files
local_first = "true"
display_manager_integration = "true"
# keep temporarily deleted files in the local Bin trash
use_recycle_bin = "true"
# latency and monitoring
## check every 10 seconds local -> server changes
monitor_fullscan_frequency = "10"
## check every minute server -> local changes
monitor_interval = "60"
```

Note that **Comments must be placed in separated lines, not inline**. Alternatively, you may use `sync_list` file, to be stored as well in `~/.config/onedrive` folder

ii. Authenticate, and run OneDrive once

```
onedrive --sync --dry-run # optional, list folders that will be synchronsied
onedrive --sync --verbose # first initialisation, usually time-consuming, one-shot
```

4. Permanent synchronization using `systemtcl` facilities. **Highly recommended to remove folders at the local level, rather than on the server**.

```
systemctl --user enable onedrive
systemctl --user start onedrive # in the back end, use onedrive --monitor
```

5. Monitor currently uploaded files:

```
journalctl --user-unit onedrive -f (continous) # equivalent to systemctl --user status onedr

onedrive --display-config
onedrive --version
```

6. Other useful OneDrive instructions:

- `onedrive --create-share-link <path/to/file>` for read-only shareable links, and `onedrive --create-share-link <path/to/file> --with-editing-perms` to add writing rights.

7. (Optional) Visual interaction of OneDrive synchronization:

i. OneDrive system tray program adds a simple icon returning the status of synchronization for OneDrive, that can be controlled with by clicking with the right button of the mouse. - You'll need to install, and run `qmake` version 5 (by default, on recent Ubuntu configurations, it's the 6th that is used, which does not compile successfully the program).

```bash
# install the 5th version of qmake
sudo apt install qt5-qmake qtbase5-dev

# explicity use the qmake version 5, by calling the entire absolute path
# usually located under folder /usr/lib/qt5/bin
# avoid switching from qmake version 6 to qmake version 5 as default
# using symbolic links, as most recent projects use version 6 of qmake
```

- Run following bash file:

```
# create the runnable executive file
git clone https://github.com/DanielBorgesOliveira/onedrive_tray.git
cd onedrive_tray
mkdir build
cd build
/usr/lib/qt5/bin/qmake ../systray.pro # prepare the configuration, such as Makefile
make # build the software

# execute it, such that it runs whenever log in
sudo make install # deploy the software, such as copying files -> here's the sudo incen
```

```
systemctl enable --user onedrive_tray.service
```

```
Source code
   ↓
qmake (.pro → Makefile)
   ↓
make (Makefile → binary)
   ↓
make install (binary → system)
   ↓
systemctl enable (autostart)
```

ii. Customise the logs output reported on the terminal, with the following script that reports in *real-time* a colourised OneDrive activity of the logs: a. Install dependencies: `sudo apt install silversearcher-ag ccze` b. Add `onedrive_log` bash file into folder `~/scripts/tools`. This script is a coloured wrapper of `journalctl` command. c. Add to the `$PATH` using local `.bashrc` the folder `~/scripts/tools`

3.2.1 Troubleshoots for OneDrive

- Call `systemctl --user restart onedrive` after modifying the `.config` file to account for changes within it. For retro-application of configuration changes, use `onedrive --resync --sync --verbose` with parsimony.
- Warning "onedrive application is already running, please check process list" means that several concurrent onedrive are running in the background:
  - List existing processes using onedrive: `ps aux | grep onedrive`
  - Force kill (-9) processes that do no not use `systemtcl`
- Configure webhooks, if you want *real-time* synchronisation with the server, `server -> local`. **Not recommended generally, as complex to configure, and usually irrelevant for daily use cases**. Instead, reduce the latency for synchronisation.
- Git and OneDrive do not interfere well for 3 reasons at least, especially when synchronising files on multiple platforms. Linux (and Mac) machines end lines with `LF (Line Feed): \n`, while Windows uses `CRLF (Carriage Return + Lien Feed): \r\n`, Linux tracks file permissions for executables while Windows doesn't (only problematic for shell scripts), finally, `.git` and `.Rproj` folders are complex binary repositories with challenging access and encoding. Three heuristic approaches to solve most issues:
  i. Exclude `.git` folders[4] (or even completely code folders) from the synchronisation. **Note that in this case, the folder will no longer be detected as tracked by `Git` processes.**

[4] Adding argument `skip_dir = ".git"` to OneDrive config file

ii. Configure globally the `.gitconfig` configuration file to allow both end of lines indicators, using `git config --global core.autocrlf input` on Linux and Mac machines collaborating with Windows machines[5], and `git config --global core.autocrlf true` on Windows machines[6]. The argument tolerates `CRLF` as it appears, otherwise use by default Linux-like `LF` endings. **This approach solves most of my synchronisation conflicts**.

iii. **Best practice**: per project, create a `.gitattributes` defining explicitly the encoding of EOL, removing the dependency to the machine OS[7]. By default, you should opt in ofr the `LF` EOL:

```
# 3 columns: file extension, file type (text, or binary), eol definition for text docum
* text=auto eol=lf # modify EOL both globally, as the default behaviour, if `core.autoc
# AND custom configuration, per file extension
*.sh   text eol=lf
*.py   text eol=lf
*.js   text eol=lf
*.ts   text eol=lf
*.java text eol=lf
```

Apply changes with the following commands:

```
git add --renormalize .
git status # display files being impacted by the change of EOL encoding
git commit -m "Normalize line endings"
```

iv. **Discard local changes**, induced by degenerate EOL diffs, with:

```
git status # Review of existing changes
git clean -fd # Optional, remove untracked files and directories as well
git reset --hard HEAD
```

to discard all local modifications to **tracked, but unstaged files**, practically bringing the working tree back to the last commit.

v. Configure the editor:

w. VS Code → `Settings > Text Editor > Files > Eol`, go for the `\n`, or `auto` option. To apply it on pre-existing files, use *Change End Of Line Sequence command*.

ii. RStudio: `Tools → Global Options → Code → Saving -> Line ending Conversion → (Posix) LF`.

[5] Accept as EOL CRLF, but always commit as LF
[6] on Windows machines, checked out as `CRLF`, but committed as `LF`
[7] Detailled information is avalaible here

Graphical Interaces for OneDrive

Avoid using graphical interfaces, such as OneDriveGUI, they poorly concur with local installation of `onedrive` on my personal experiment.
Instead, rely on the terminal, as it usually provides the user with better customisation, and control on OneDrive configuration.

3.3 Monitor processes in the long-run

- `systemctl list-unit-files --type=service --state=running --user` to list all processes running in the background, even after rebooting, restrained to the user-level.
  - Alternatively, processes at the system-level are stored under `/lib/systemd/system/`
- `systemctl status <service>` retrieves current status

4 Software to install

**General recommendation*: avoid using `ChatGPT`, as the instructions for installing software are often deprecated and/or not following Ubuntu best guidelines.

Install software on Ubuntu, some guidelines

Three recommended methods, depending on Ubuntu's support:

1. If supported by Ubuntu, go for `apt install` for system software[8]:

- *automatic updates* and security patches -> **it's the only method allowing it in a systematic way**.
- easiest method for installation
- signed and verified packages
- recommended for *core system*, long-term software
- If **not directly available on Ubuntu's native package manager, or using in the back-end `snap` commands**, switch to pre-built tarball installation[9], or add the repository (only if **trusted**) to `apt allowed keys` folder. Example for `Docker`, Section 4.6, or custom Zotero installation file.
- **.deb files** are intended for Linux distributions derived from Debian (such as Ubuntu, Linux Mint, etc.), while **.rpm files** are mainly used by distributions derived from Red Hat–based systems (such as Fedora, CentOS, and RHEL) + `openSUSE` distribution:
  - No auto-updates
  - Potential dependency issues
  - **Always install with `sudo apt install`, rather than `sudo`**

`dpkg -i`, to better solve dependencies, fix broken installs, and ensures system consistency.

---

2. `AppImages` are standalone executable files, bundling the application and most of its dependencies:

- Recommended for *desktop applications* to be installed only on the user account -> does not require `sudo` rights in most cases
- Portable
- Works across many distributions
- `AppImageLauncher` can help in organising all AppImages within the same folder + integrate them into the system menus:
  - Relevant if using multiple AppImages
  - Using menu integrations and icons
  - Avoid `Snap`

---

3. Pre-built tarball (`.tar(gz|xz)`) archives:

- Stronger customisation
- Works without distribution packages
- May fail when built / requires manual handling/no automatic updates
- **Only recommended for developers, or if nothing else exists**.

- And now, the not recommended installations:
  - `snap` stores pre-compiled versions of tools, but are less recommended compared to standard installation with `sudo apt install`
  - `FlatPak+Flathub`

Conclusion: APT first (if not directly available, add the repository to the trusted keys of `apt`) -> AppImage second -> `.deb` if AppImage is not available -> source build in the worst case.

Feature	AppImage	tar/build	apt install	.deb
Root required	no	sometimes	yes	yes
Dependency handling	Bundled	Manual	Automatic	Partial
Auto-updates	no	no	yes	no
System integration	Low	Medium	High	High
Portability	Excellent	Low	None	Low
Stability	App-dependent	Variable	Excellent	Good
Recommended for beginners	yes	no	yes	

- Other choices of importance:
  - `portable`, or not. Portable installation is not managed by `apt`, easier to remove (does not leave leftovers/footprints, as only installed locally, in the home directory), but does easily enable desktop integration (with icons, for instance), nor allows for automatic updates
  - Non-portable applications are usually better for daily/intensive use, but not *sandboxed*, and harder to remove (configuration files installed in root directories)

4.1 Shell-customisation (including `$PATH`)

4.1.1 Zsh (advanced bash configuration)

```
sudo apt update
sudo apt install zsh

# define this shell shebang by default
chsh -s $(which zsh) # modify /etc/passwd repository
```

4.1.2 Add executables

- System-wide:
  - `/usr`, and manual installs under `/usr/local` are in the `$PATH` by default
  - For third-party GUI, *self-contained* apps stored under `/opt`, add a symlink: `sudo ln -s /opt/<appName> /usr/local/bin/<appName>`, requires **`sudo` rights**.
- User-level:
  - Customise `.bashrc` (or `.zshrc`) configuration files to modify the `$PATH` -> **Only affects the user, and interactive shell**. Examples:

```
export PATH="$PATH:$HOME/scripts/tools"
alias ll='ls -lah'
```

  - AppImages may be integrated using AppImageLauncher, or `.desktop` configuration.
- Deal with multiple versions of the same program (for instance, `Java`): use `update-alternatives --list <AppName>` to list executables.

[9] `apt-get` works similarly, but is now deprecated

[9] Fetch the latest stable archive, extract it, and add executables to your `PATH`, see example under Section 4.13

4.2 AppImageLauncher

```
# download the .deb archive, with the proper architecture configuration
# https://github.com/TheAssassin/AppImageLauncher/releases/tag/v3.0.0-beta-3

sudo apt install appimagelauncher_3.0.0-beta-2-gha287.96cb937_amd64.deb

# update dependencies
sudo apt update
sudo apt install -f
```

- For deleting apps installed with AppImage Launcher, you'll need to delete both *Applications* folder, and `~/.local/share/applications/` that stores executables.

4.3 PDF management

4.3.1 Sioyek as PDF Viewer

`Sioyek` is an open-source PDF viewer focusing on technical papers with mathematical formulas. Major uses cases are coloured *highlights* of text sections with `h` shortcut, smart jump to figures by clicking on them, and bookmark text sections with shortcut `b`.

1. Choose between portable (for isolated environment), or standard distribution under Sioyek releases
2. Unzip the archive, within it, make it executable with `chmod +x`.
3. Run it

4.3.2 Sejda for editing PDFs in an interactive way

- Web version
- Sedja *add-on* for Chrome

4.3.3 PDFtk

- To edit PDFs, if you prefer the command line.

4.4 Image Editors (to replace Paint on Windows)

Sorted from closest experience to Paint, to most generalist:

1. `Drawing`

   i. Add Drawing to the `apt` repository of allowed PPAs: `sudo add-apt-repository ppa:cartes/drawing`
   ii. Install running `sudo apt install drawing`

2. `Libre Office Draw`

3. `Gimp` (closer equivalent to `Photoshop`):

```
sudo apt install gimp
sudo apt install gimp-plugin-registry # add extensions
```

4.5 LibreOffice

1. Install main software (such as Libre Office Write, or Calc) with `sudo apt install libreoffice libreoffice-gtk3`
2. Relevant extensions, that you can add using `Tools -> Extensions`

i. Rotate Images in free text documents with *Rotate* extension
ii. Install dictionaries + extension Grammalecte and LanguageTool[10]. *needs to check if LanguageTool is truly relevant, is AI-based, but also resource-consuming.*

```
# French dictionary
sudo apt update
sudo apt install libreoffice-l10n-fr # add French hyphenation
```

iii. Install Latex extension

4.6 Docker

1. Download dependencies, configuration files and add repository to apt for automatic updates:

```
sudo apt update
sudo apt install ca-certificates curl # dependencies for downloading packages, and verifying

# Part I: create, download and make Docker apt key readable for verifying its integrity
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Part II: define where, and which version of Docker, should be donwloaded + how to verify i
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc # folder previously defined
EOF

# Part III: update with newer configuration
sudo apt update
```

[10] You may configure the Java version used by default under `Tools -> Options -> Advanced`

2. Install Docker toolkit with `sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin`
3. Check installation with `sudo docker run hello-world`. If not running by default upon login, enable automated start with `sudo systemctl start docker`. More details here.

4.7 Zoom

1. Retrieve latest version, choosing proper Linux distributions and OS architecture under Zoom downloads
2. Run `sudo apt install ./zoom_amd64.deb`
3. Log-in using your university email address, using `SSO`. The account is provided with unlimited meetings available by default, use following domain: `univ-amu-fr`. [11]

4.8 Teams

- **Use the web version, as Microsoft no longer supports .deb versions for Linux.**

4.9 Firefox

Avoid running directly `sudo apt install firefox` -> it installs by default `snapd`, and will use not stable, beta `Snap` version of Firefox.

1. Import, and add Mozilla APT signing key:

```
# Download APT key
wget -q https://packages.mozilla.org/apt/repo-signing-key.gpg -O- | sudo tee /etc/apt/keyrin

# Add it to the Apt registry
cat <<EOF | sudo tee /etc/apt/sources.list.d/mozilla.sources
Types: deb
URIs: https://packages.mozilla.org/apt
Suites: mozilla
Components: main
Signed-By: /etc/apt/keyrings/packages.mozilla.org.asc
EOF

# Provide higher priority to Mozilla Firefox

echo '
```

[11] Alternatively, use the web version available here

```
Package: *
Pin: origin packages.mozilla.org
Pin-Priority: 1000
' | sudo tee /etc/apt/preferences.d/mozilla
```

2. Install Firefox using Mozilla provider:

```
sudo apt update
sudo apt install firefox

which firefox # should not report `snapd` folder
apt policy firefox
```

4.10 Slack

1. Download Slack application
2. Run it with `sudo apt install slack-desktop-4.47.69-amd64.deb`
3. Update with:

```
sudo apt-get update
sudo apt-get upgrade slack-desktop
```

4.11 Zulip

1. Run following instructions:

```
sudo apt install curl # if not already installed
sudo curl -fL -o /etc/apt/trusted.gpg.d/zulip-desktop.asc \
    https://download.zulip.com/desktop/apt/zulip-desktop.asc
echo "deb https://download.zulip.com/desktop/apt stable main" | \
    sudo tee /etc/apt/sources.list.d/zulip-desktop.list
sudo apt update
sudo apt install zulip
```

2. Log-in (recommended with your Github profile) on each of the channels you want to connect, to keep notified about newest posts.

4.12 Cytoscape

Major default: only supported by Java version 17 (and not by more recent, and stable Java versions!!)

1. Install Java version 17 with `sudo apt install openjdk-17-jdk`[12]
2. Follow this tutorial

i. Retrieve the latest version

[12] On Ubuntu, a more modern version of Java, the 21, was installed

ii. Make it executable with `chmod u+x <filename.sh>`
iii. Execute with `sh`, or simply `./` (avoid using `sudo` if irrelevant for other users of the computer)
iv. Launch with `Cytoscape &`.

4.13 Zotero

1. `sudo apt install libreoffice-java-common -y` adds Java dependencies to seamlessly integrate Zotero with Libre Office *plug-in* (for instance, automated retrieval of bibliographic references).

2. Retrieve the latest stable version on https://www.zotero.org/download/[13].

3. Run following instructions + Synchronise by installing Zotero Connector on your website:

```
#!/bin/bash

# part I: extract archive
sudo tar -xjf zotero.tar.bz2 -C /opt
sudo mv /opt/Zotero_linux-x86_64 /opt/zotero

# Part II: add the executable
sudo ln -s /opt/zotero/zotero /usr/local/bin/zotero
```

4. (Optional: add a custom Desktop, and launcher icon for an executable) Create file `~/.local/share/applications/zotero.desktop`, with the following content. Make the launcher executable with `chmod +x ~/.local/share/applications/zotero.desktop`

```
[Desktop Entry]
Name=Zotero
Exec=/usr/local/bin/zotero
Icon=/opt/zotero/icons/icon128.png
StartupWMClass=Zotero
Type=Application
Terminal=false
Categories=Office;
MimeType=text/plain;x-scheme-handler/zotero;application/x-research-info-systems;text/x-resea
X-GNOME-SingleWindow=true
```

5. Inspired from Zotero hacks tutorial, enable unlimited synced storage for articles on multiple machines.

i. Add required plug-ins, `Better BibTeX` (mandatory for custom expert as a plain .bib file) + `Zotmoov` (**optional**, replace Zotfile for renaming files, for

[13] Alternatively, download with `curl`

example, sorting articles by authors' names).

ii. Edit Zotero preferences[14]: - Uncheck the option to create automatic web page snapshots (increases cluttering with lots of small files added) - Uncheck full-text sync - Set apart in `Edit->Settings->Advanced` the
   - Base directory (should be the folder synced by your File sharing software), with
   - Custom Data directory location. This local folder should be located in a different position, and will be managed by Zotero itself

`![Zotero Database Configuration](zotero-advanced-configuration.png)`

iii. Edit `Better BixTex` *plug-in*: - Personally, use `[auth:lower][year][journal:lower:abbr]` - Should be the same across machines.

4.14 Quarto, R and RStudio

1. To get the latest R Versions, add the signing key, along with the repository:

```
sudo mkdir -p /etc/apt/keyrings
sudo wget -O /etc/apt/keyrings/cran.gpg https://cloud.r-project.org/bin/linux/ubuntu/marutte
echo "deb [signed-by=/etc/apt/keyrings/cran.gpg] https://cloud.r-project.org/bin/linux/ubunt
```

2. Install R:

```
sudo apt update
sudo apt install r-base r-base-dev -y
R --version
```

3. Install RStudio (Desktop IDE)

i. Go to: https://www.rstudio.com/products/rstudio/download/#download/ Choose the **Ubuntu/Debian `.deb` package** for your architecture (amd64).

ii. Step 2 — Install the `.deb` package:

```
sudo apt install ./rstudio-2025.09.0-422-amd64.deb
```

3. Install Quarto, Go to https://quarto.org/docs/get-started/

```
sudo apt install ./quarto-1.3.496-linux-amd64.deb
quarto check
```

4.15 Latex

```
sudo apt install texlive-full -y
# other commonly used packages
sudo apt install texlive-latex-extra texlive-fonts-recommended texlive-science biber latexmk
```

[14]*What matters the most here is uniformity across platforms

4.16 WhatsApp

- No supported local Linux installation
- Instead, recommended to use the Web version of WhatsApp, then pair it with your mobile device.

4.17 Cursor

1. Download DEB extension.
2. Run it:

```
sudo apt install ./cursor_2.0.77_amd64.deb
```